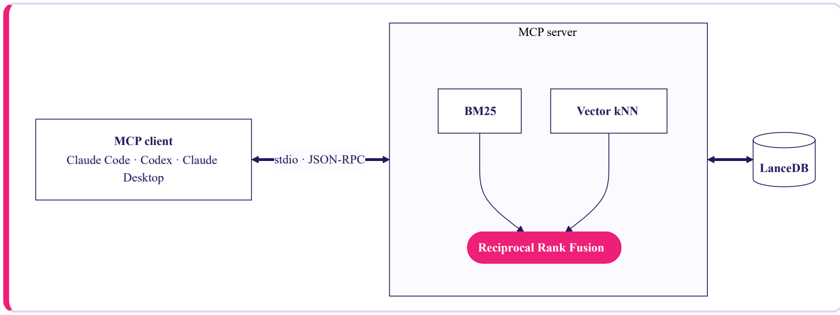
MCP-CUSTOMER-SUPPORT-TICKETS

MCP server for a 62k-ticket support corpus.

AUTHOR Tal Charnes **STACK** Python · FastMCP · LanceDB **DATASET** Tobi-Bueck/customer-support-tickets · ~62k · EN+DE
TRANSPORT stdio · local



One Python process. BM25 and vector kNN run in parallel; **Reciprocal Rank Fusion** merges by rank position and returns the top 10 with stable `ticket://{id}` citations. Rows, BM25 index, and 384-dim `e5-small` vectors live in one LanceDB table.

01 Design

Three layers in one Python process. **Ingest** pulls the Hugging Face dataset, derives `sha1(revision‖row_index)` ids stable across machines, and embeds rows with `e5-small` (384-dim, multilingual, on-device). **Store** is a single LanceDB table holding rows, the BM25 index, and the vectors. **Serve** exposes six tools (`search_tickets`, `get_ticket`, `aggregate_tickets`, `create` / `update` / `delete_ticket`), two resources (`ticket://{id}`, `schema://tickets`), and one prompt — `draft_reply` — which assembles the target ticket plus five grounding neighbours; the *client's* LLM writes the reply.

02 Why this solution

Hybrid beats either retriever alone — BM25 catches literal tokens (error codes, SKUs); vectors catch paraphrase; RRF fuses by rank position, so there are no weights to tune. LanceDB co-locates rows, BM25, and vectors in one embedded store that scales to 100M+ on the same API. **Avoided:** cross-encoder rerank (marginal once an LLM re-reads snippets), HyDE / query rewriting, chunking (tickets are already short), remote HTTP + OAuth, and any server-side LLM call — no API-key dependency, no hallucinated tickets.

03 In production

Live ticket source with incremental delta-ingest. Streamable HTTP transport with OAuth 2.1 and tenant-partitioned vector indices. `bge-reranker-base` on by default once retrieval feeds real workflows; `bge-m3` embeddings on a small GPU for richer multilingual coverage (incl. Hebrew). PII redaction at ingest. An eval harness of 50–200 golden queries tracking precision@5 per deploy. Traces, p95 dashboards, per-tenant rate limits; signed images, CVE gating, and pinned model artifacts.

04 Risks

Hallucinated tickets. Read endpoints return dataset bytes verbatim; the server never calls an LLM. **Prompt injection via ticket bodies.** Content wrapped in `<ticket>` tags; every tool description ends with a reminder that text inside those tags is data, not instructions. **Dataset drift.** HF revision pinned into every `ticket://` URI and the embedding cache key; re-index triggers on revision change. **Confident answers from weak matches.** `search_tickets` returns body snippets so the LLM judges from the text, not the rank; `draft_reply` refuses to scaffold unless a prior ticket clears cosine ≥ 0.70 with a non-empty answer; unsupported filters fail loud (no timestamp column).

05 Business impact

The same code, pointed at your own support corpus, lets reps and CSMs ask plain-English questions inside Claude or Cursor — *"how did we resolve the last few issues about X?"* — and get an answer linked back to the actual tickets. Institutional knowledge stops living only in individual reps' heads. **Swap the corpus and the pipeline doesn't change:** account-review notes, incident write-ups, internal runbooks — same retrieval, same citation contract. **Because the interface is MCP, one server is reachable from every AI surface** — Claude Code for engineers, Cursor and Codex for product, any internal agent — with no per-tool integration work. Build the retrieval surface once; every AI client gets it.